RECAP -- PIPELINING HAZARDS

- Hazards prevent next instruction from executing during its designated clock cycle, thus limiting the speedup
  - Structural hazards: HW cannot support this combination of instructions (single person to fold and put clothes away)
  - Data hazards: Instruction depends on result of prior instruction still in the pipeline (matching socks in later load)
  - Control hazards: conditional branches & other instructions may stall the pipeline -- delaying later instructions (must check detergent level before washing next load)

CONTROL HAZARDS

- What is the fundamental operating principle of pipelining?
  - ***Fetch and Execute new Instruction on every Cycle***.

- However pipeline can't always fetch/execute ***correct*** instruction.
  - We have already seen the Structural and Data Hazards.

Instr	Cycle 1	Cycle2	Cycle 3	Cycle4	Cycle5	Cycle 6	Cycle 7	Cycle 8	Cycle 9
Lw	IF	ID	EX	MEM	WB				
Add	-	IF	ID	EX	MEM	WB			
Sub			IF	ID	EX	MEM	WB		
STALL				Bubble	Bubble	Bubble	Bubble	Bubble	
And				STALL	IF	ID	EX	MEM	WB

- What is control dependence?
  - To determine which instruction to execute next...

Control Dependence and Hazards

- **Control Dependence**: What is the correct address of the next instruction (PC)?
  - It is equivalent of **True Data dependence** on **Program Counter**

- ***Question: In Von-Neuman Architecture: All instructions are control dependent on previous ones. Why?***

Branch Types

Instruction Type	Branch decision at IF/ID time	# of possible next fetch addresses?	Next fetch requires	When is next fetch address resolved?
Conditional Branch Example: ***beq $s1 $s2 loop*** ***bne $s1 $s2 loop***	Unknown	2 IF(TRUE) go to loop; ELSE do {*blah blah*}	Instruction and Data	Execution stage (register dependent)
Unconditional Example: ***J 2500***	Always taken	1	Instruction	Decode stage (PC + offset)
Call Example: ***Jal 2500***	Always taken	1	Instruction	Decode stage (PC + offset)
Return Example: ***Jr $ra***	Always taken	Many	Instruction and Data	Execution stage (register dependent)

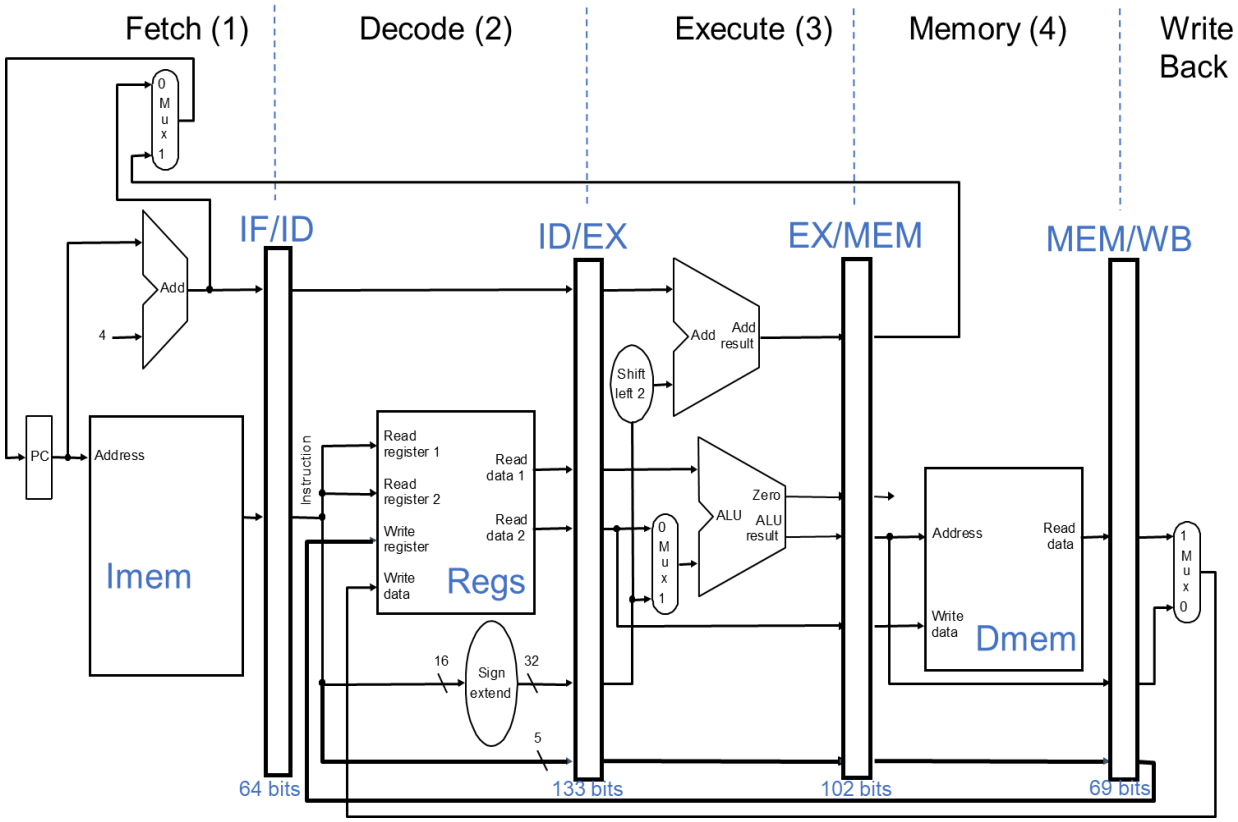
BRANCHING – FORWARD AND BACKWARD BRANCHES

High-level code

```
uint32_t sum=0, i =0, n=10;
for (i=0; i<n; i++) {
    Sum +=i;
}
printf("Sum of  %d… ");
```

MIPS assembly code

```
  add  $s1, $0, $0      # sum = 0
  add  $s0, $0, $0      # i   = 0
  addi $t0, $0, 10      # $t0 = 10
for:
  beq  $s0, $t0, done   # branch on i=10
  add  $s1, $s1, $s0    # sum = sum + i
  addi $s0, $s0, 1      # increment i
  j    for
done:
  syscall print
```



Branching – Forward and Backward Branches

High-level code

```
sum=0;
i =0;
n=10;
for (i=0; i<=n; i++) {
    Sum +=i;
}
printf("Sum of  %d… ");
```

MIPS assembly code

```
  add  $s1, $0, $0       # sum = 0
  add  $s0, $0, $0       # i   = 0
  addi $t0, $0, 10       # $t0 = 10
for:
  beq  $s0, $t0, done    # if i == 10, branch
  add  $s1, $s1, $s0     # sum = sum + i
  addi $s0, $s0, 1       # increment i
  j    for
done:
  syscall print
```

2021 Mid-term question

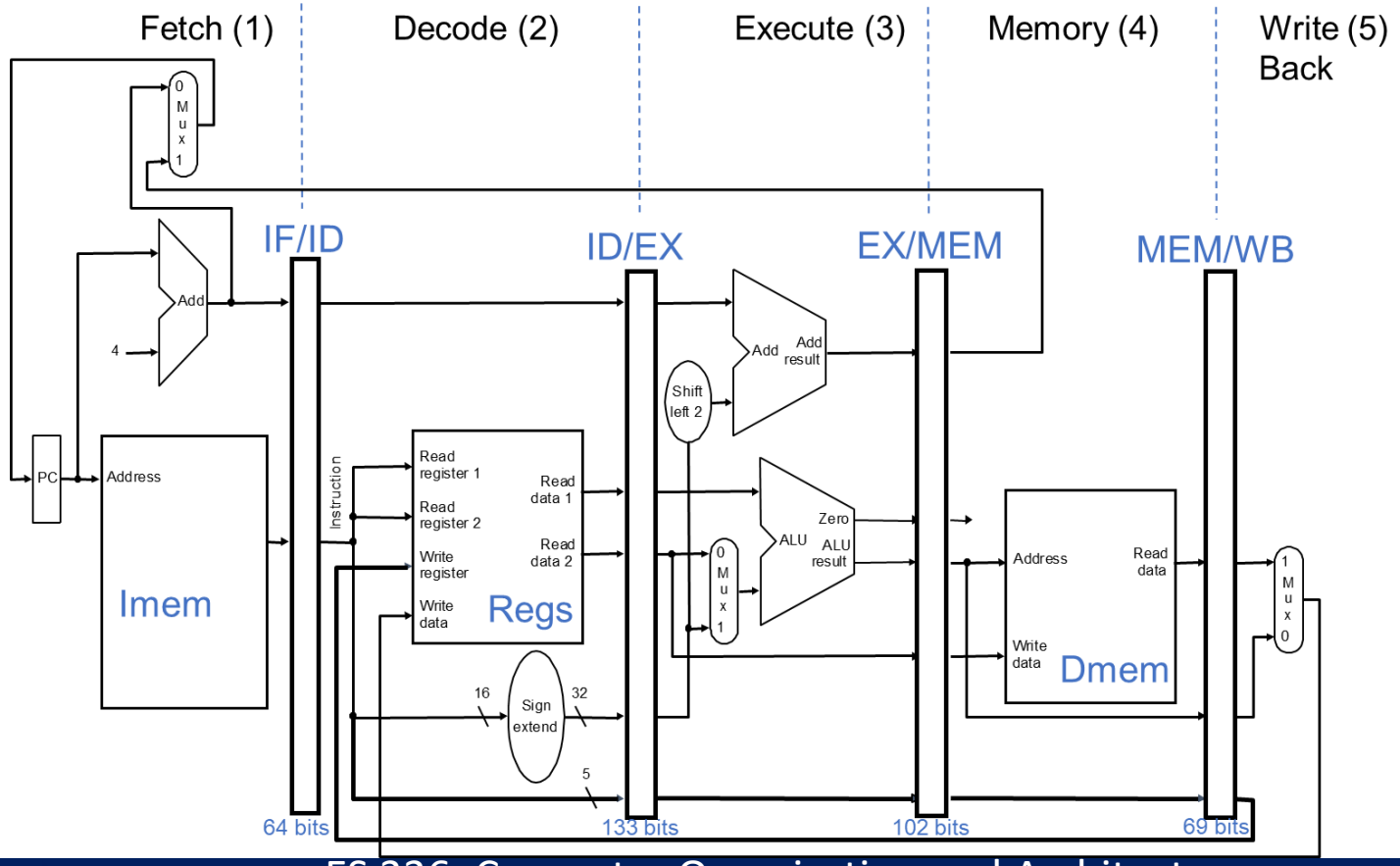
```
arr[10]
i =0;
n=10;
for (i=0; i<n; i++) {
    if(arr[i] &0x01) arr[i]+=1;
}
```

```
for: //assume $s1, t1 = baseaddr and arr[0]
  beq  $s0, $t0, end     # if i==10, end; lw $t1
  andi $s2, $t1, 1       # $s2 = arr[i] & 1
  beq  Ss2, $0, noincr   # if arr[i] is even
  addi $t1, $t1, 1       # make odd to even; sw
noincr:
  addi $s0, $s0, 1       # increment i
  addi $s1, $s1, 4       # go to next elem
  j    for
```

How to Handle Control Dependences

Potential solutions if the instruction is a control-flow instruction:

- Stall the pipeline until we know the next fetch address
- Employ delayed branching (branch delay slot)
- Guess the next fetch address (branch prediction)

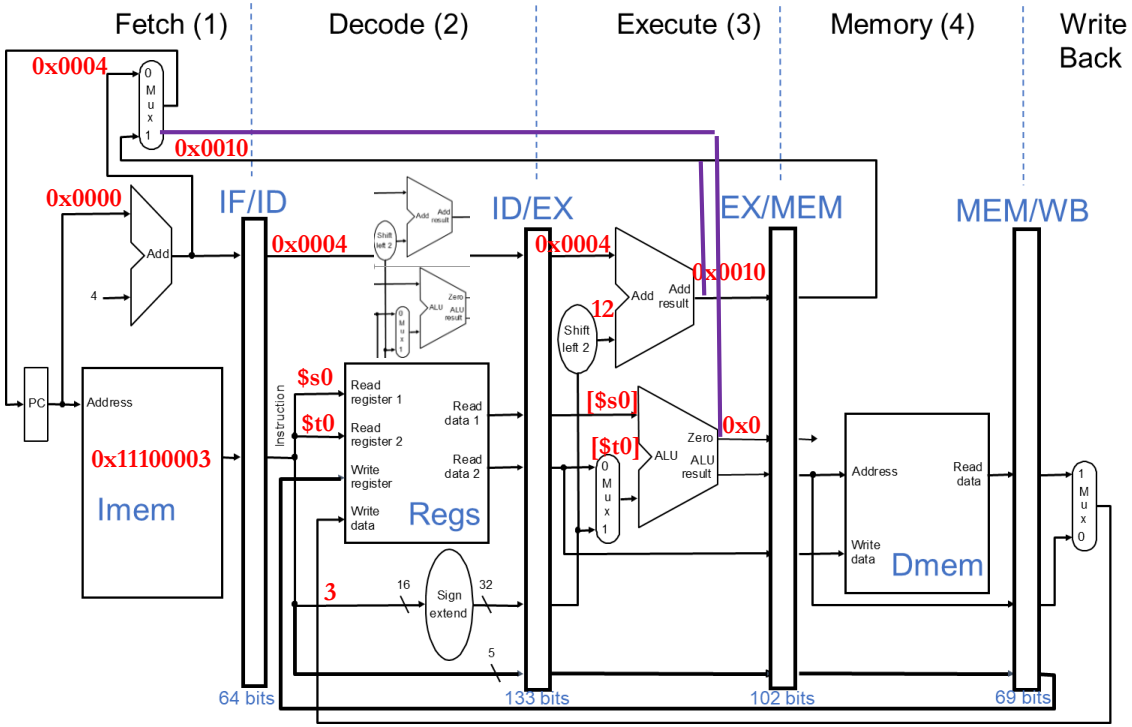


STALL ON BRANCH

Must wait until branch outcome is resolved to fetch the next instruction

```
for:
  beq  $s0, $t0, done
  add  $s1, $s1, $s0
  addi $s0, $s0, 1
  j    for
done:
  syscall print
```

Instr	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5
BEQ	IF (I1)	ID (I1)	EX (I1)	MEM (I1)	WB (I1)
STALL		Bubble1	Bubble1	Bubble1	Bubble1
STALL			Bubble2	Bubble2	Bubble2
STALL				Bubble3	ID (I2)
addi					IF (I3)



- In our MIPS Pipeline design, We would need **3** or **2 stalls**!
  - Next Ins. IF can begin only after completion of EX stage.
- By adding extra HW, we can at-best reduce to **1 stall**
  - i.e. by computing BTA and BC in ID stage.

Issues with Stall on Branch Approach?

- Rather than waiting for true-dependence on PC to resolve, just guess nextPC = PC+4 to keep fetching every cycle

  Is this a good guess?

  What do you lose if you guessed incorrectly?

- ~20% of the instruction mix is control flow

  Overall, typically ~70% taken and ~30% not taken [Lee and Smith, 1984]

  - ~50 % of "forward" control flow (i.e., if-then-else) is taken
  - ~90% of "backward" control flow (i.e., loop back) is taken

- Expect "nextPC = PC+4" ~86% of the time, but what about the remaining 14%?

Performance Analysis (Next PC = PC+4)

- correct PC $\Rightarrow$ no penalty 86% of the time
- incorrect PC $\Rightarrow$ 2 bubbles (baseline) 14% of the time
- Assume
  - no data hazards
  - 20% control flow instructions
  - 70% of control flow instructions are taken

CPI = [1 + (0.20*0.7) * 2] = [1 + 0.14 * 2] = 1.28

```
  add  $s1, $0, $0        # sum = 0
  add  $s0, $0, $0        # i   = 0
  addi $t0, $0, 10        # $t0 = 10
for:
  beq  $s0, $t0, done     # if i == 10, branch
  add  $s1, $s1, $s0      # sum = sum + i
  addi $s0, $s0, 1        # increment i
  j    for
done:
  syscall print
```

probability of a wrong Insn.

penalty for a wrong Insn.

What if we reduce the penalty to 1 cycle?

CPI = [1 + 0.14 * 1] = 1.14

Can we reduce either of the two penalty terms?

How to Handle Control Dependences

Potential solutions if the instruction is a control-flow instruction:

- Stall the pipeline until we know the next fetch address
- Employ delayed branching (branch delay slot)
- Guess the next fetch address (branch prediction)

- Similarly Load (Load delay slot)

DELAYED BRANCHING

- Change the semantics of a branch instruction
  - Branch after **N** instructions/**N** cycles

- Basic Idea: Delay the execution of a branch. N instructions (delay slots) that come after the branch are always executed regardless of branch direction.

- Problem: How do you find instructions to fill the delay slots?
  - Branch must be independent of delay slot instructions

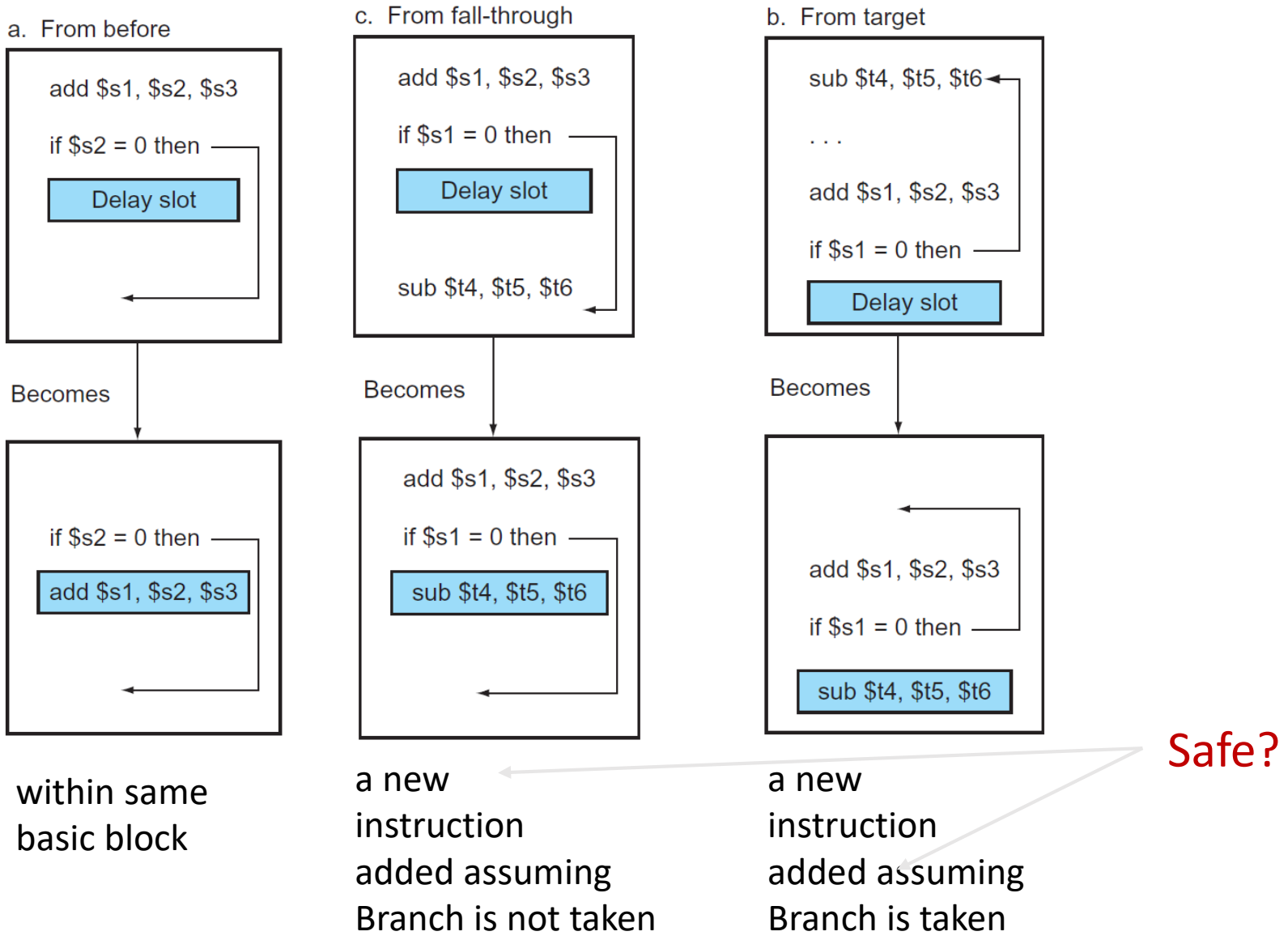
- ***Unconditional branch***: Easier to find instructions to fill the delay slot

- ***Conditional branch***: Condition computation should not depend on instructions in delay slots → ***difficult to fill the delay slot***

Filling the Delay Slot

Basic Idea: Reorder Data Independent Instructions which do not change the program semantics.

```
  add  $s1, $0, $0
  add  $s0, $0, $0
  addi $t0, $0, 10
for:
  beq  $s0, $t0, done
  add  $s1, $s1, $s0
  addi $s0, $s0, 1
  j    for
done:
  syscall print
```



within same basic block

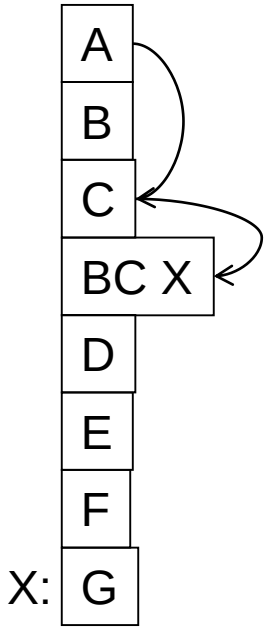
a new instruction added assuming Branch is not taken

a new instruction added assuming Branch is taken

Safe?

DELAYED BRANCHING -- USING 2-STAGE PIPELINE

Normal code:



Timeline:

if	ex
A	
B	A
C	B
BC	C
--	BC
G	--

6 cycles

Example:

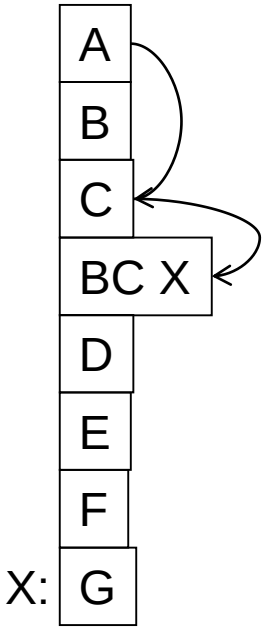
```
A:   add r1, r2, r3
B:   add r4, r5, r6
C:   add r1, r1, r7
BC:  beq r5, r6, X
G:   add r8, r9, r10
```

```
A:   add r1, r2, r3
C:   add r1, r1, r7
BC:  beq r5, r6, X
B:   add r4, r5, r6
G:   add r8, r9, r10
```

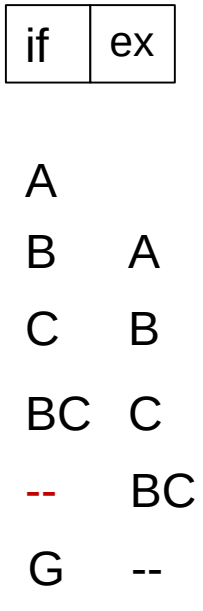
```
A:   add r1, r2, r3
BC:  beq r5, r6, X
B:   add r4, r5, r6
C:   add r1, r1, r7
G:   add r8, r9, r10
```

Delayed Branching

Normal code:

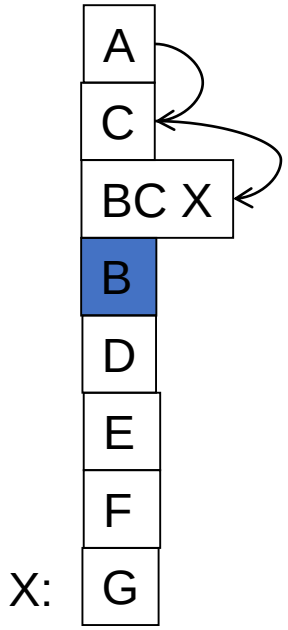


Timeline:

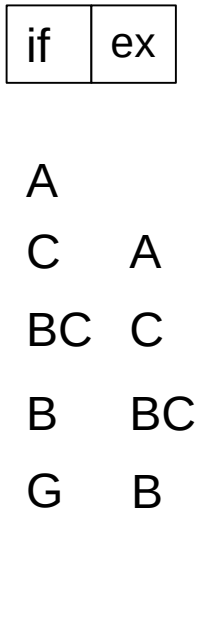


6 cycles

Delayed branch code:



Timeline:



5 cycles

Example:

```
A:  add r1, r2, r3
B:  add r4, r5, r6
C:  add r1, r1, r7
BC: beq r5, r6, X
G:  add r8, r9, r10
```

```
A:  add r1, r2, r3
C:  add r1, r1, r7
BC: beq r5, r6, X
B:  add r4, r5, r6
G:  add r8, r9, r10
```

```
A:  add r1, r2, r3
BC: beq r5, r6, X
B:  add r4, r5, r6
C:  add r1, r1, r7
G:  add r8, r9, r10
```

Delayed Branching

- Advantages:

  + Keeps the pipeline full with useful instructions in a simple way assuming

    1. All delay slots can be filled with useful instructions

    2. Number of delay slots == number of instructions to keep the pipeline full before the branch resolves.

- Disadvantages:

  -- Not easy to fill the delay slots (even with a 2-stage pipeline)

    1. Number of delay slots increases with pipeline depth, superscalar execution width
    2. Number of delay slots would vary for variable latency operations.

  -- Ties ISA semantics to hardware implementation

    -- SPARC, MIPS: 1 delay slot

    -- What if pipeline implementation changes with the next design?

How to Handle Control Dependences

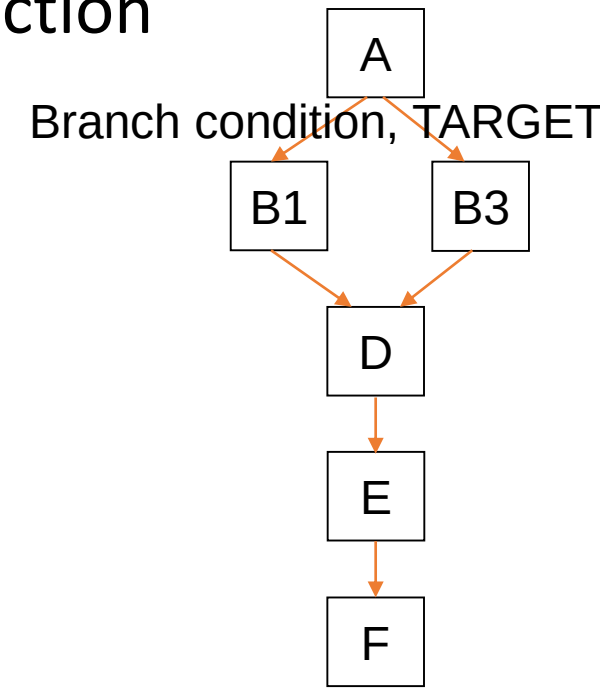
Potential solutions if the instruction is a control-flow instruction:

- Stall the pipeline until we know the next fetch address
- Employ delayed branching (branch delay slot)
- Guess the next fetch address (branch prediction)

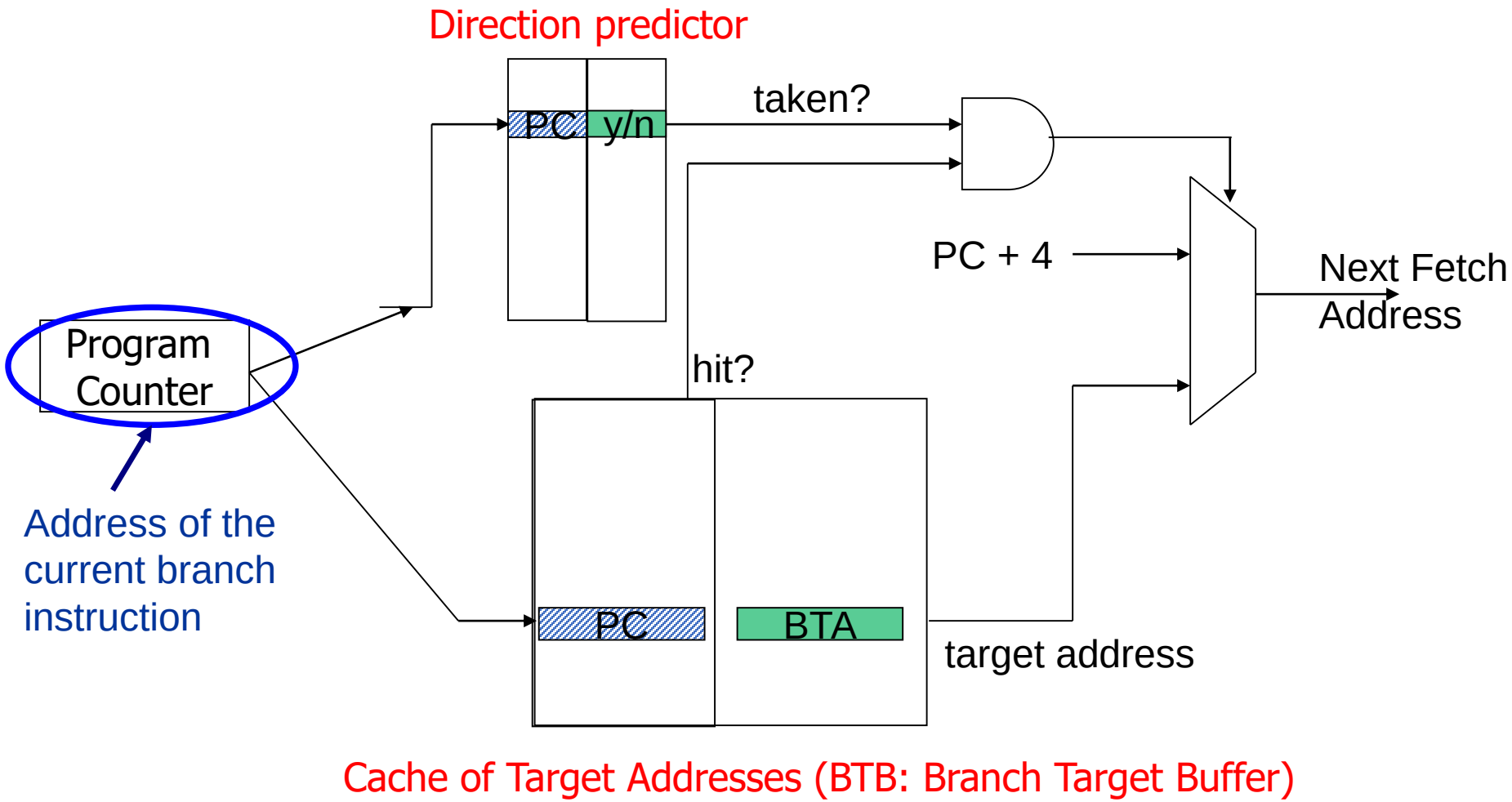
BRANCH PREDICTION

How do we keep the pipeline full in the presence of branches?

- Predict the next instruction address when a branch instruction is fetched
- Requires two kind of information:
  - i) Direction of the Branch ← Predict with some intelligence.
  - ii) Associated Target of a branch ← Compute from Instruction
- One more critical information:
  - i) Is this a branch instruction? ← Identify from the PC.



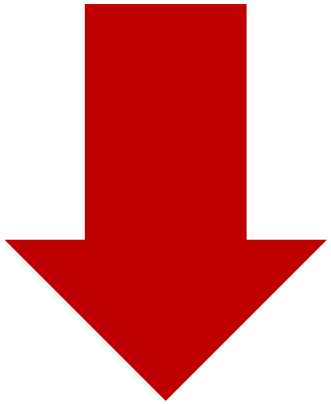
Instruction Fetch (IF) Stage with BTB and Direction Prediction



Branch Direction Prediction

Compile time (Static)

- Always not taken (ANT)
- Always taken (AT)
- Backward taken, forward not taken (BTFN)
- Profile/Program analysis
- Programmer Hint based (likely direction)



Overheads:

Power

Price

Performance

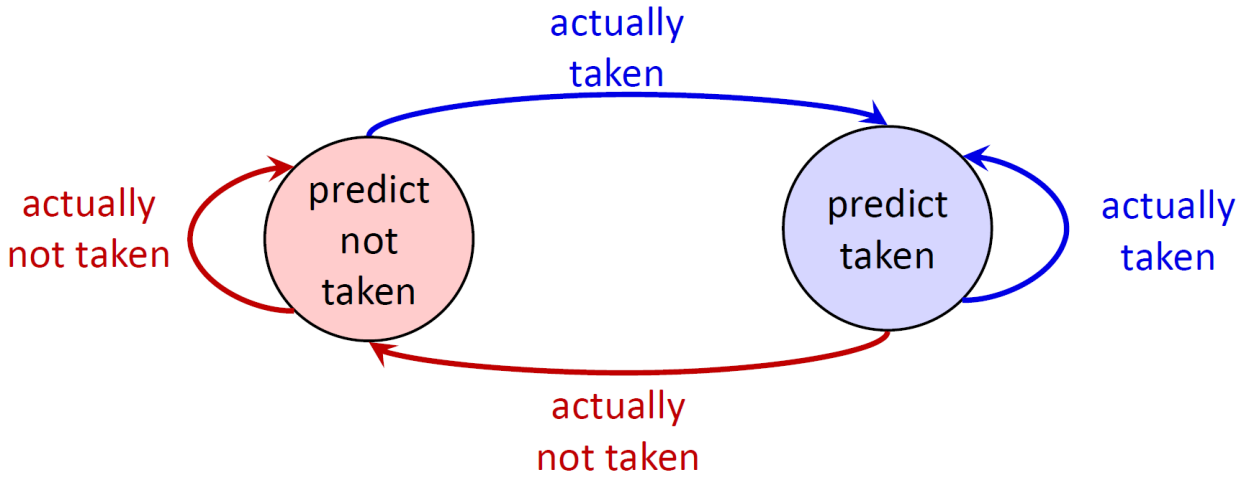
Run time (Dynamic)

- Last time prediction (single-bit) – ***1-bit BBP***
- N-bit counter based prediction -- ***2-bit-SC/HC***
- Two-level prediction (global vs. local) -- ***G-Share***
- Hybrid (counter and level) -- ***Tournament***
- Neural Network based -- ***Perceptron***

Accuracy

1-Bit (Bimodal) Branch Predictor

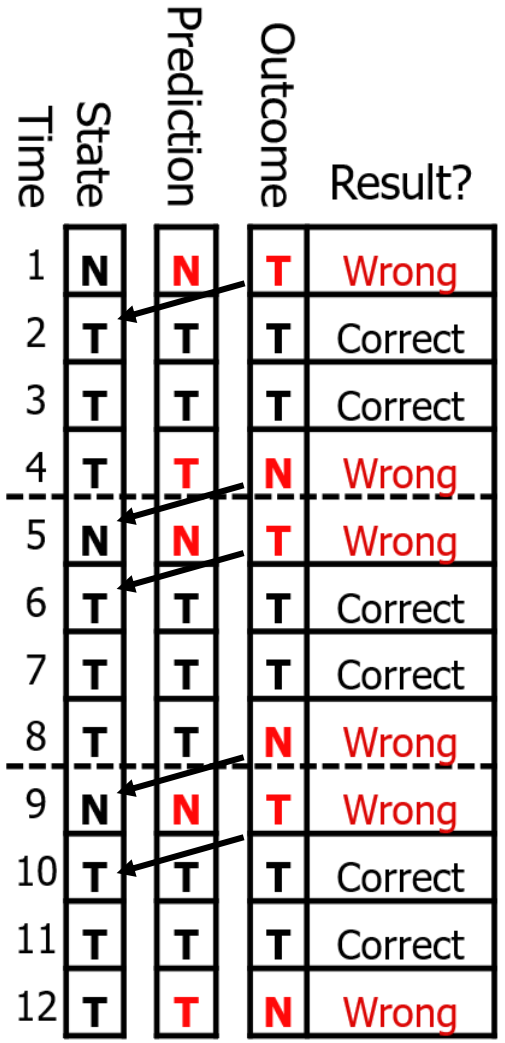
- Very simple 1-bit direction predictor (0 = N, 1 = T)
  - Essentially: branch will go same way it went last time



  - Problem: **inner loop branch** below

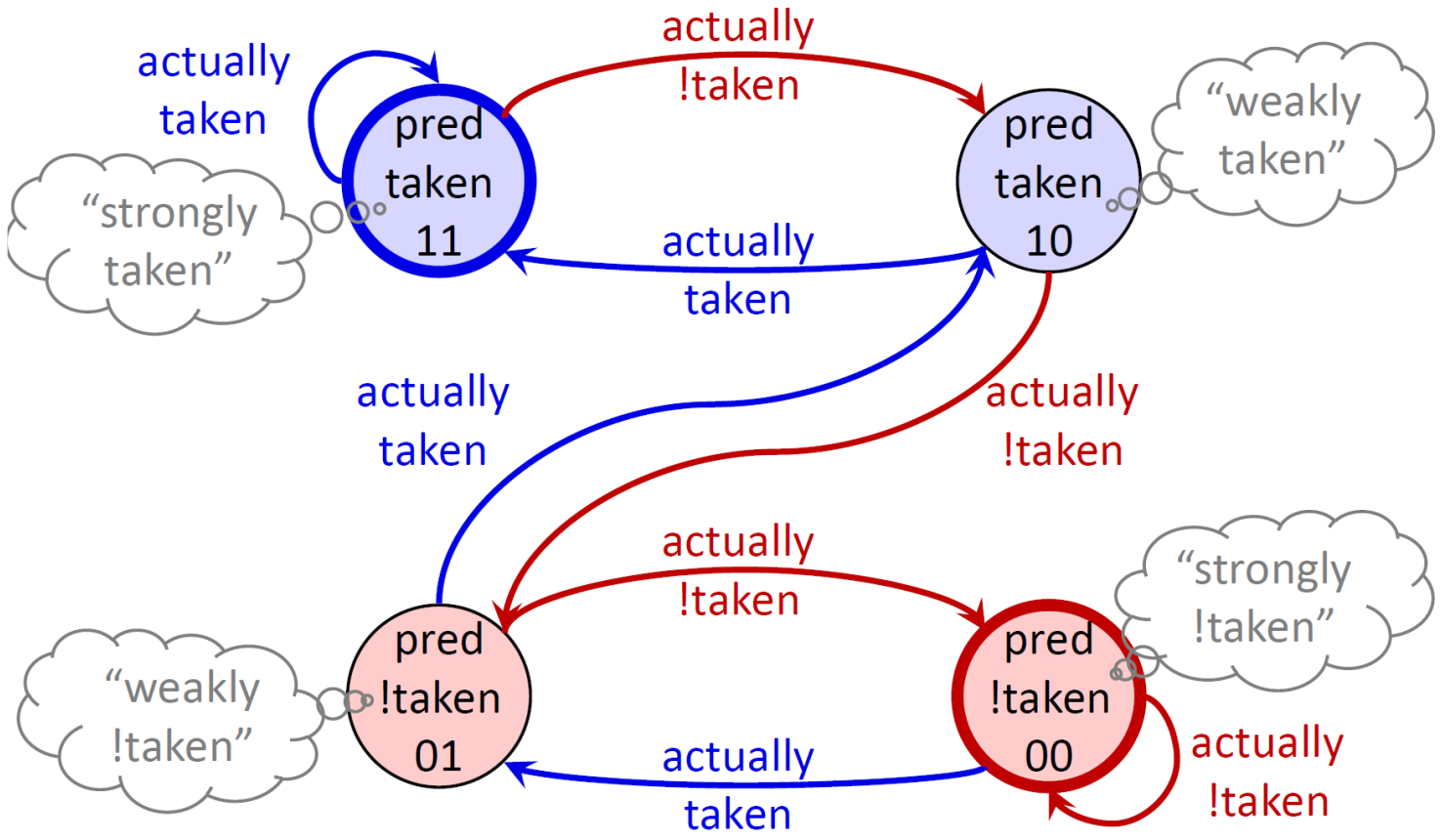
```
for (i=0;i<100;i++)
    for (j=0;j<3;j++)
        // some instructions
```

    - Two "built-in" mis-predictions per inner loop iteration
    - Branch predictor "changes its mind too quickly"



Two-Bit Saturating Counters (2bc)

- **Two-bit saturating counters (2bc)** [Smith 1981]
  - Replace each single-bit prediction: (0,1,2,3) = (N,n,t,T)
  - One misprediction for each loop execution




```
for (i=0;i<100;i++)
    for (j=0;j<3;j++)
        // some instructions
```

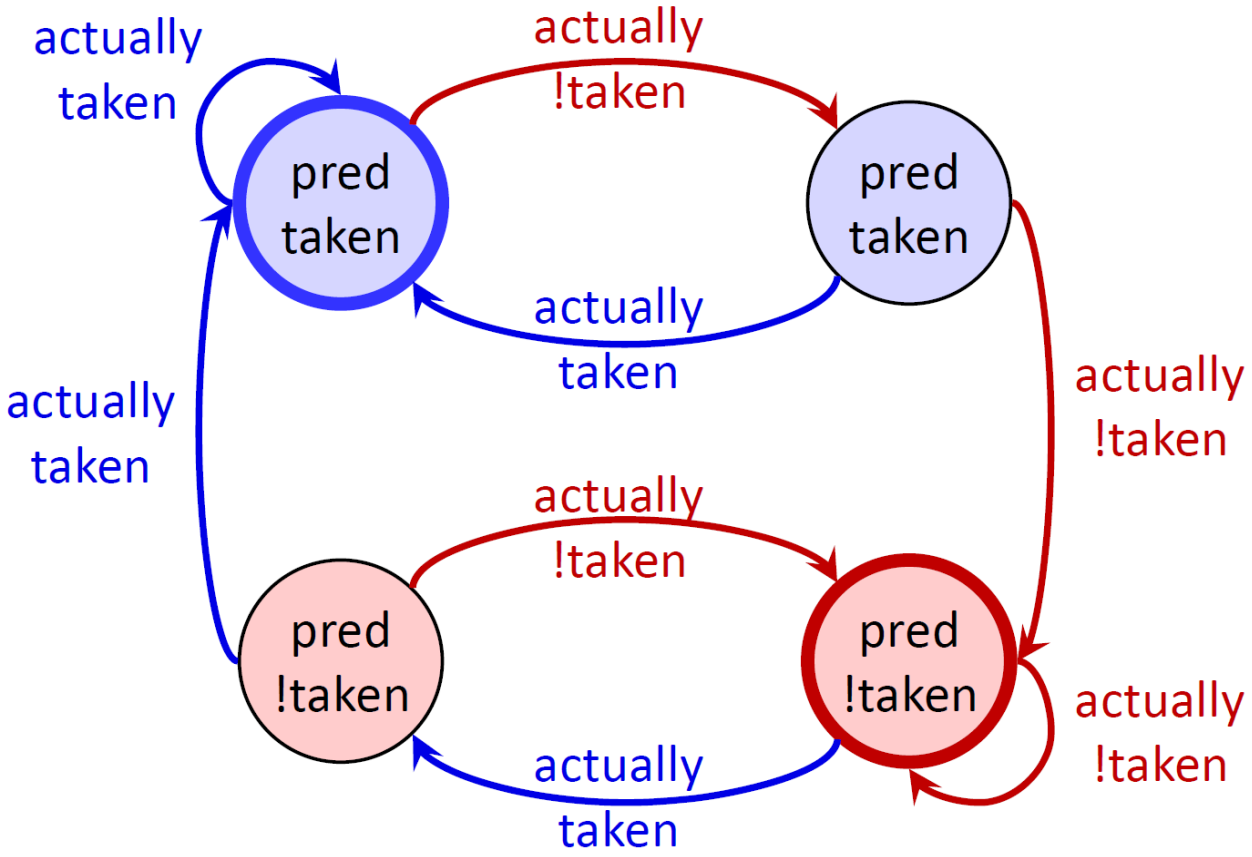
Time	State	Prediction	Outcome	Result?
1	N	N	T	Wrong
2	n	N	T	Wrong
3	t	T	T	Correct
4	T	T	N	Wrong
5	t	T	T	Correct
6	T	T	T	Correct
7	T	T	T	Correct
8	T	T	N	Wrong
9	t	T	T	Correct
10	T	T	T	Correct
11	T	T	T	Correct
12	T	T	N	Wrong

- Can we do even better?

Two-Bit Saturating Counters (2bc)

- **Two-bit Hysteresis counters**
  - Same as before: (0,1,2,3) = (N,n,t,T)
  - Instead of saturation, use history of previously taken decision
  - Force predictor to mis-predict twice before "changing its mind"

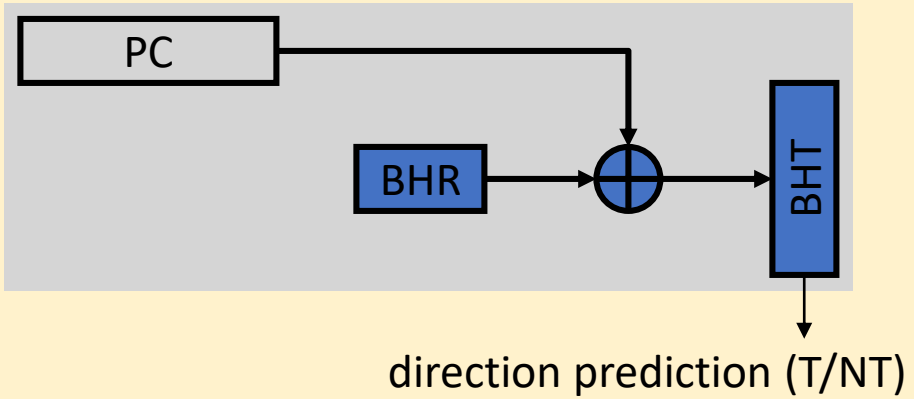
```
for (i=0;i<100;i++)
   for (j=0;j<3;j++)
      // some instructions
```



BRANCHES MAY BE CORRELATED

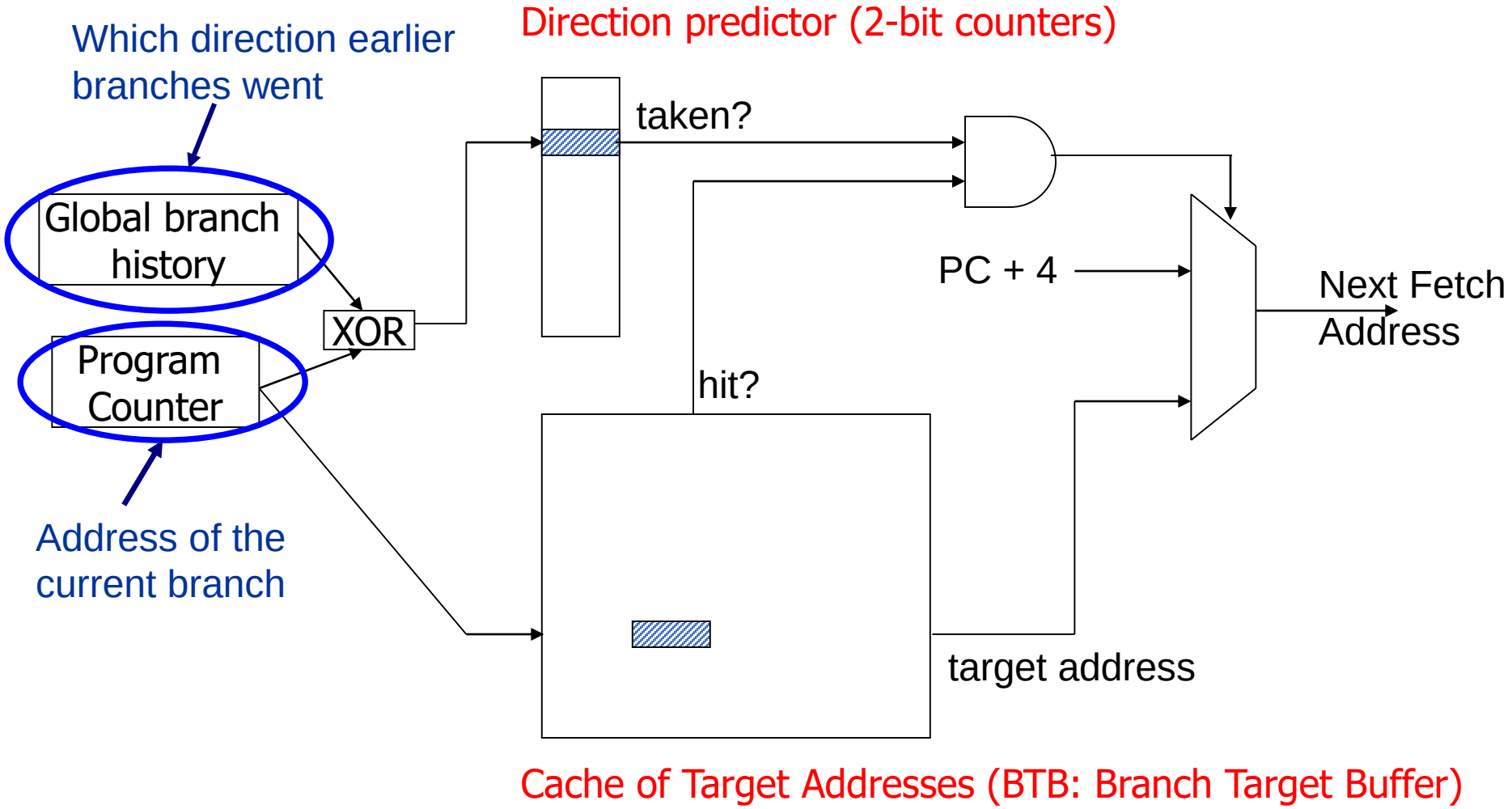
- Consider:

```
for (i=0; i<1000000; i++) {    // Highly biased
    if (i % 2 == 0) {          // Locally correlated
        … y = random()%3 …
    }
    if (y == 0) {   // Unpredictable
        …
    }
    if (i % 2 == 0) {
        …              // Globally correlated
    }
}
```



- Exploits observation that branch outcomes are correlated
- How do we incorporate history into our predictions?
  - Use PC xor BHR to index into BHT. Why?
- Maintains recent branch outcomes in **Branch History Register (BHR)**
  - In addition to BHT of counters (typically 2-bit sat. counters)

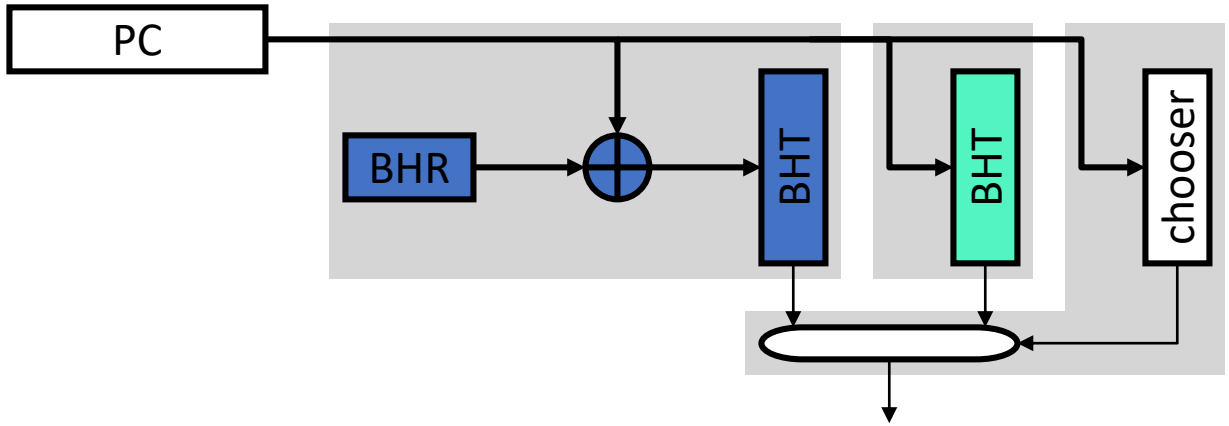
More Sophisticated Branch Direction Prediction



Tournament Predictor

- **Tournament (Hybrid) predictor** [McFarling 1993]
  - Idea: combine two predictors
    - **Simple bimodal predictor** for history-independent branches
    - **Correlated predictor** for branches that need history
    - **Chooser** assigns branches to one predictor or the other
    - Branches start in simple BHT, move mis-prediction threshold

+ 90–95% accuracy

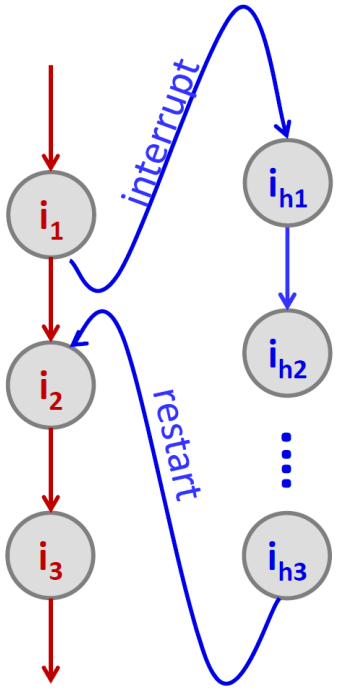


More Branch Predictor works and competitions:
https://www.jilp.org/vol13/index.html

EXCEPTIONS and INTERRUPTS

Interrupts and Exceptions

Form of Control Hazard: Any *event* that requests the attention of the processor
-- An unplanned Function call to a third-party routine



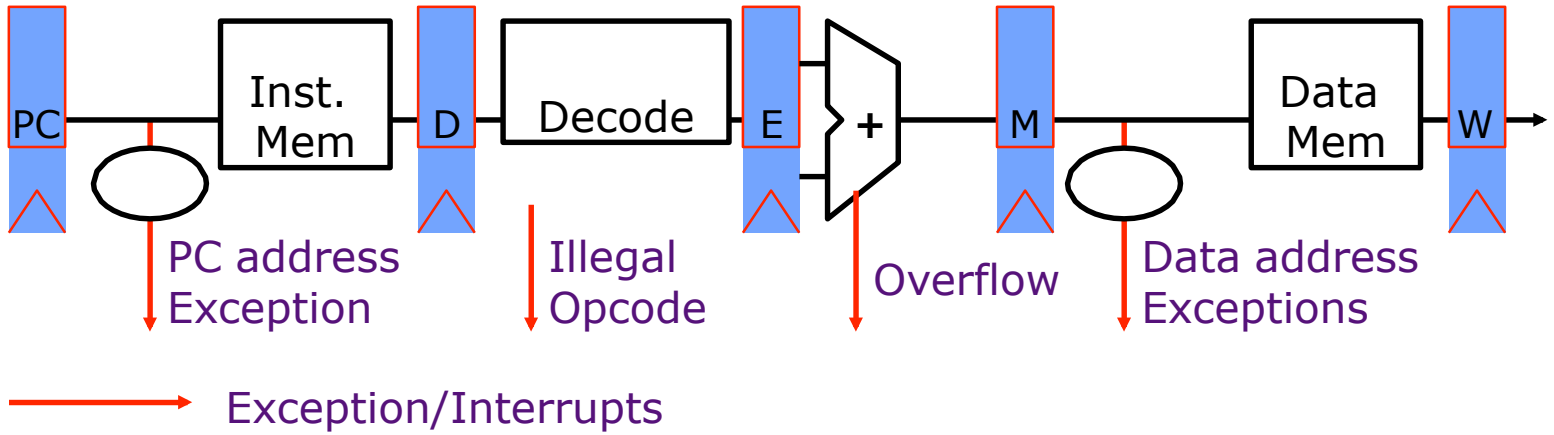
An *external or internal event* that needs to be processed by the system (another program).

From the program's point of view: such an event is usually unexpected.

Exceptions vs. Interrupts

- Cause
  - Exceptions: internal to the running thread
  - Interrupts: external to the running thread
- When to Handle
  - Exceptions: when detected (Synchronous)
  - Interrupts: when convenient (Asynchronous)
    - Except for very high priority ones like
      - Power failure/Machine check
- Handling Context: process (exception), system (interrupt)
- Priority: process (exception), depends (interrupt)

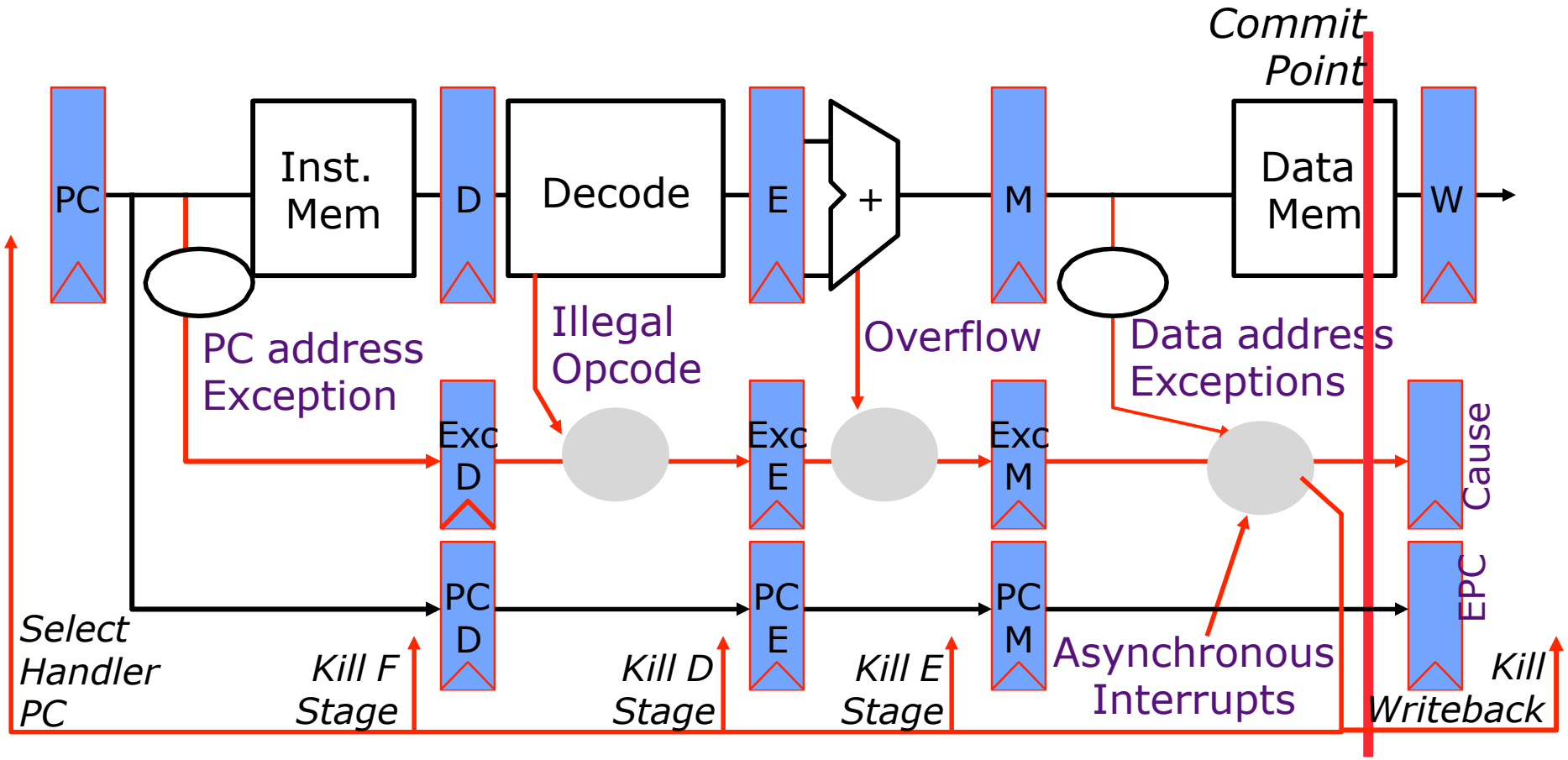
Exception Handling in 5-stage pipeline



- **IF:**
  - I-mem address/protection fault
- **ID:**
  - illegal opcode
  - syscall instruction (SW requested exception)
  - trap to SW emulation of unimplemented instructions
- **EX:**
  - invalid results: overflow, divide by zero, etc.
- **MEM:**
  - D-mem address/protection fault
- **WB:** ??
  - nothing can stop an instruction now…

- How to handle multiple simultaneous exceptions in different pipeline stages?
- How and where to handle external asynchronous interrupts?

Exception Handling in 5-stage pipeline



PRECISE EXCEPTIONS/INTERRUPTS

- The architectural state should be consistent when the exception/interrupt is ready to be handled

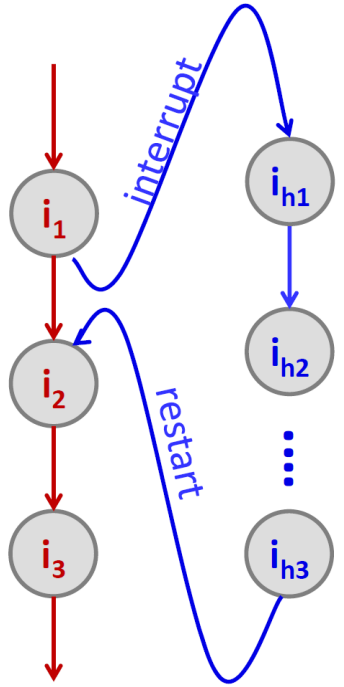
1. All previous instructions should be completely retired.

Retire = commit = finish execution and update arch. state

2. No later instruction should be retired.

MIPS INTERRUPT ARCHITECTURE

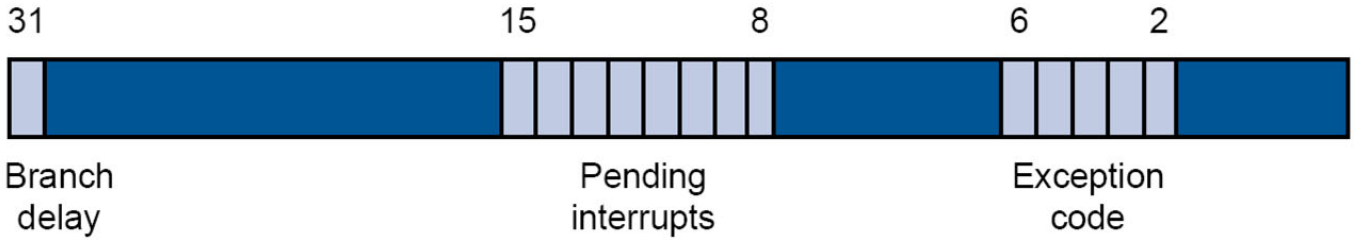
- On interrupt transfer, HW saves interrupted address to a special **EPC** register
  - can't just leave in PC: overwritten immediately
  - can't use GPR: need to preserve user value



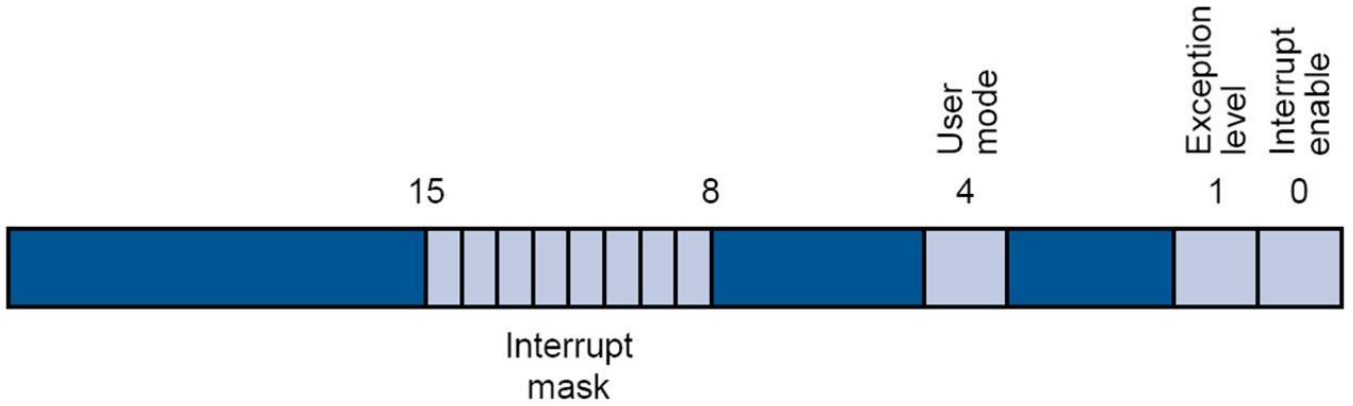
- For example, GPR can be preserved by interrupt handler using callee-saved convention
  - MIPS convention reserves the usage of **r26** and **r27** for the small interrupt handlers

MIPS INTERRUPT ARCHITECTURE

- Privileged system control registers; loaded automatically on interrupt transfer events
  - **EPC** (CR14): exception program counter, → determines which instruction location to go back to
  - **Cause** (CR 13): → determines what caused the interrupt



  - **Status** (CR 12): → enable/disable interrupts, set privilege modes



- Accessed by "move from/to co-processor" instructions

HANDLING INTERRUPTS

- **Option 1:** control transfers to default handler
  - default handler examines CR12 & CR13 to select specialized handler

- **Option 2:** vectored interrupt
  - separate specialized handler addresses registered with hardware
  - hardware transfer control directly to appropriate handler to reduce interrupt processing time

Note: handler in address region/space protected from user so user can't just branch to it unprivileged user also can't imitate handler code